

TALL TALE PLATFORM

Codex Implementation Specification

Cinematic Role Gateway | Player Library | Captain Command
Tall Tale Studio | Invitations | Immutable Historical Playthroughs

PRIMARY BUILD OUTCOME

Transform the existing Player, Captain, and Studio pages into a single, secure, versioned Tall Tale platform that feels like entering a game and preserves every player voyage as a lasting historical record.

Prepared for implementation in the existing project repository

July 17, 2026

01 HOW CODEX MUST USE THIS SPECIFICATION

This document is the implementation prompt. Treat it as an executable product and engineering contract, not as loose brainstorming.

CODEX OPERATING INSTRUCTION

Read this entire specification before editing files. Inspect the existing repository, current route structure, authentication behavior, database models, API boundaries, animation system, and design language. Preserve working functionality. Adapt the architecture to the actual stack instead of replacing the project with a disconnected rewrite.

Required working behavior

- **Work from the current repository state.** Do not assume a framework, database, deployment target, or component library until the repository proves it.
- **Create an implementation inventory first.** Identify the existing Player page, Captain page, Studio page, current password behavior, Tall Tale records, active playthrough state, media storage, and any current authentication/session logic.
- **Build incrementally.** Introduce migrations, compatibility adapters, and route redirects where necessary. Do not destroy current Tall Tales or invalidate current Player and Captain URLs without a safe transition path.
- **Use server-enforced authorization.** Hiding buttons is presentation only. Every protected page, API request, asset fetch, mutation, realtime channel, and export must be authorized on the server.
- **Prefer polished completeness over scaffolding.** Do not leave placeholder buttons, dead navigation, fake status text, hard-coded sample progress, or unimplemented invitation actions.
- **Verify the complete user journey.** Run tests and manually exercise the role gateway, Player sign-in, Captain sign-in, Creator entry, invitation creation, invitation acceptance, voyage start, progress continuation, completion, and historical replay views.

Expected deliverables from Codex

- [] Repository-aware implementation plan recorded in the project documentation.
- [] Database migrations and seed/backfill logic that preserve existing Tall Tales and playthroughs.
- [] Role gateway and mode-specific authentication flows.
- [] Player Tall Tale Library with invited, ready, active, completed, and historical playthrough states.
- [] Captain Library, invitation wizard, invitation management, and active-voyage launch controls.
- [] Creator/Studio entry and role-aware workspace switching.
- [] Immutable published Tale versions and immutable playthrough-version binding.
- [] Server-side role, resource, field, and asset authorization.
- [] Responsive, accessible, cinematic frontend behavior with reduced-motion support.
- [] Automated unit, integration, authorization, and end-to-end tests.
- [] Updated developer documentation, route map, data model documentation, and migration notes.

02 PRODUCT VISION AND NON-NEGOTIABLE PRINCIPLES

The platform must feel like a deliberate adventure experience while behaving like a disciplined multi-role application underneath.

Product vision

The website must become the central Tall Tale platform. A visitor first enters through a cinematic role gateway, then receives a completely different experience depending on whether they enter as Player, Captain, or Creator. The system underneath must remain unified so Tall Tales, published versions, playthroughs, invitations, assets, and history are never duplicated across disconnected applications.

The six non-negotiable principles

Principle	Required interpretation
One platform, three experiences	Player, Captain, and Creator use shared domain data but receive different navigation, permissions, copy, and workflows.
Role selection is not authorization	Selecting Captain or Creator only opens that authentication path. It never grants access.
The Player owns a library experience	Players can see every Tall Tale they were invited or added to, every active voyage, every completed voyage, and the exact historical version associated with each completed playthrough.
Published history is immutable	A playthrough remains bound to the exact published version used when it began. Later edits cannot rewrite the player's past.
Spoilers are protected as data	Future chapters, hidden assets, Captain instructions, unrevealed titles, logic conditions, and private notes must never be returned to Player clients before authorization and reveal conditions permit them.
Cinematic does not mean dishonest	Animations may be elaborate, but statuses, progress, presence, invitation state, and synchronization indicators must reflect real backend truth.

PRODUCT SMELL TO PREVENT

Do not create three separate databases, three separate Tall Tale copies, or three unrelated application shells. The entire purpose of the library is to make every experience feel connected while permissions determine what each person is allowed to see and do.

03 INFORMATION ARCHITECTURE

The library is the organizing center; the role gateway is the front door; Player, Captain, and Creator are role-specific workspaces.

Top-level route families

```
/
/player
/player/sign-in
/player/library
/join/:invitationToken
/play/:playthroughId
/captain
/captain/sign-in
/captain/library
/captain/tales/:taleId
/captain/voyages/:playthroughId
/captain/invitations
/studio
/studio/sign-in
/studio/library
/studio/tales/:taleId
/studio/tales/:taleId/versions
/studio/tales/:taleId/publish
```

The exact path syntax may follow the current framework, but the conceptual route families must remain distinct. The Player application must not contain visible Captain or Studio navigation. Captain and Creator may share an authenticated account and switch authorized workspaces, but the Player experience must remain fully isolated.

Workspace relationships

Workspace	Primary purpose	Default landing area	May expose
Public Gateway	Choose entry intent	Role selection scene	Player, Captain, Creator entry choices
Player	Experience and remember voyages	Player Tall Tale Library	Only player-safe Tall Tale and playthrough data
Captain	Operate voyages and manage players	Captain Library / Command	Authorized Tale controls, invitations, player progress, Captain notes
Creator	Author and publish content	Studio Library	Drafts, media, logic, validation, versions, publishing

Tall Tale details page

Every Tall Tale must have a central details page appropriate to the current workspace. The underlying Tale identity is shared, while the visible tabs and actions vary by role.

- **Player details:** spoiler-safe overview, invitations, active run, completed runs, memory archive, exact version played, collected media, revealed artifacts, and replay availability.
- **Captain details:** overview, published versions, active voyages, invited players, completed playthroughs, readiness, Captain preview, launch actions, and operational history.
- **Creator details:** draft health, chapters, block graph, media, logic, tests, publishing, version history, comparisons, restore/fork operations, and playthrough compatibility warnings.

04 CINEMATIC ROLE SELECTION GATEWAY

The first page must feel like starting a game, not choosing a department from a corporate intranet.

Opening scene

Build a full-screen, responsive opening gateway that introduces the world before asking for credentials. The preferred scene is a dim captain's cabin or weathered chart table emerging from darkness and ocean ambience. The visual treatment must match the established Tall Tale art direction and must not introduce a generic fantasy theme.

1. Begin with darkness and subtle ocean or ship ambience, respecting prior audio consent and browser autoplay restrictions.
2. Reveal a horizon glow, lantern light, or moving water reflection.
3. Bring the role objects into view through layered motion rather than a single opacity fade.
4. Allow interaction as soon as the controls are understandable; do not make the visitor endure an unskippable cinematic every visit.
5. Remember returning sessions and provide a fast path while preserving a visible Return to Role Selection control outside the Player experience.

Role objects and interaction

Role	Visual object	Supporting copy	Interaction behavior
Player	Open journal, compass, or sealed invitation	Continue an adventure, accept an invitation, or revisit completed voyages.	Pages stir, compass responds, invitation seal glows.
Captain	Ship wheel, chart table, key, or command lantern	Guide voyages, invite players, reveal clues, and control the journey.	Lantern brightens, charts align, navigation marks appear.
Creator	Quill, ink bottle, drafting tools, loose maps	Create, test, publish, and preserve Tall Tales.	Ink draws structures and sketches into the map.

Two-role fallback

Implement the data and permission model for all three roles even if the current design temporarily displays only Player and Captain. If only two buttons are shown, the authenticated Captain workspace must provide an explicit Open Studio or Switch to Creator Workspace action for accounts with Creator permission. Do not collapse Creator authorization into frontend visibility assumptions.

Returning visitor behavior

- **Returning Player:** show Continue [Tall Tale Name] when exactly one active voyage exists, or Open My Tall Tale Library when multiple items require selection.
- **Returning Captain:** show Return to Captain's Command and a truthful summary such as one player awaiting launch or two invitations pending.
- **Returning Creator:** show Continue Editing [Tall Tale Name] when a recent draft is known, plus Open Studio Library.
- **Expired session:** retain the selected intent and return the user to the matching sign-in screen without exposing protected details.

Animation requirements

- Use layered transforms, lighting, depth, cloth/paper response, water reflection, and object-specific motion. Avoid a ten-second opacity fade pretending to be cinematic.
- Provide skip, reduced-motion, and replay controls where appropriate.
- Do not block keyboard navigation or screen-reader access while visual animation runs.
- Pause or simplify expensive effects when the tab is hidden, the device is low-power, or reduced motion is enabled.
- Ensure the gateway reaches a stable, usable state after navigation back, refresh, session restore, and browser history traversal.

05 AUTHENTICATION, SESSIONS, AND WORKSPACE INTENT

Authentication identifies the person; authorization determines the role; workspace intent determines which authorized interface opens.

Player authentication

Preserve the current simple Player password experience while evolving it into a real Player identity and invitation model. A Player may enter through an invitation link, short invitation code, remembered session, or direct Player sign-in. The platform must support lightweight access without requiring every invited person to create a full email-based account immediately.

- **Invitation entry:** the token identifies a specific invitation and playthrough. Ask only for the configured PIN/password or identity confirmation.
- **Direct Player sign-in:** support Player display name/username plus password, or the current password-only flow when the player identity is already established by URL, device, invitation, or existing session.
- **Guest Player profile:** allow a playthrough-bound profile that can later be claimed by a full account without losing history.
- **Remembered session:** use secure, revocable server-side sessions or securely signed session tokens. Do not store raw passwords, invitation tokens, or authorization claims in local storage.

Captain and Creator authentication

Captain and Creator access must use stronger account authentication than a lightweight Player PIN. Use the repository's existing authentication stack where present. Otherwise implement username/email plus password with modern password hashing, secure cookies, login throttling, and a future-compatible authentication-method table for passkeys or external providers.

Workspace intent rules

Selected entry	Authentication result	Authorized destination
Player	Valid Player identity or invitation	Player Library or invitation acceptance flow
Captain	Account with Captain permission	Captain Library / Command
Captain	Authenticated account without Captain permission	Safe access-denied page; no Captain data
Creator	Account with Creator permission	Studio Library
Creator	Account with Captain but not Creator permission	Safe no-Studio-access message and return to authorized workspace
Any protected route	No valid session	Matching sign-in screen with safe return path

CRITICAL AUTHORIZATION RULE

Never trust role, Tale ID, playthrough ID, player ID, chapter ID, asset ID, or invitation state supplied by the browser. Resolve every permission from the authenticated server-side principal and current database state.

06 AUTHORIZATION AND SPOILER-SAFE DATA ISOLATION

Security must operate at route, resource, action, field, asset, and realtime-channel levels.

Role and resource model

Capability	Player	Captain	Creator	Administrator
View authorized Player library	Yes	Preview only	Preview only	Yes
View Captain workspace	No	Assigned Tales/voyages	Optional by permission	Yes
Operate playthrough	Player actions only	Assigned voyages	No unless also Captain	Yes
Create/edit draft	No	Optional	Assigned Tales	Yes
Publish version	No	Optional policy	Authorized creator/publisher	Yes
View authoring version history	No	Authorized Tale only	Authorized Tale only	Yes
View completed version played	Yes, for own run	Assigned Tale/run	Authorized Tale/run	Yes
Manage invitations	Accept/decline own	Authorized voyages	No unless also Captain	Yes

Player-safe projection

Create a deliberate Player-safe data projection instead of serializing complete Tall Tale objects and hiding fields in React/Vue/Svelte templates. The Player API must return only information currently allowed for that player and playthrough.

- Do not return future chapter content, unrevealed chapter names, hidden location names, future artifact records, solution text, Captain notes, internal tags, branching conditions, unpublished media, or authoring metadata.
- Do not include hidden asset URLs in HTML, preload manifests, source maps, CSS variables, serialized page state, or network responses.
- Authorize every media request. A guessed image URL must not bypass reveal state.
- Use public-safe slugs or opaque IDs for Player routes. Never rely on sequential IDs as protection.
- For unauthorized protected resources, prefer a safe 404 or generic access-denied response that does not confirm the existence of hidden Tales or assets.

Required middleware/policy checks

```
authorize(principal, action, resource) must verify:
1. principal is authenticated for the requested workspace
2. principal has the required global or Tale-scoped role
3. resource belongs to an authorized Tall Tale or playthrough
4. requested action is valid for the current resource state
5. field-level reveal rules permit returned data
6. asset/realtime/export scope matches the same authorization decision
7. the decision is logged when it changes story state or access
```

Security controls

- Use secure, HttpOnly, SameSite cookies when compatible with the deployment topology.
- Use CSRF protection for cookie-authenticated mutations.

- Hash passwords with Argon2id or the strongest supported modern algorithm in the existing stack.
- Hash invitation tokens at rest; store only a prefix or identifier for support display.
- Rate-limit sign-in, invitation redemption, code lookup, password reset, and token validation.
- Rotate sessions after authentication and privilege changes.
- Revoke active sessions when a Player is removed from a playthrough or a Captain assignment is withdrawn, according to policy.
- Do not place secrets or authorization claims in analytics, logs, query strings, or client error reports.

Threat scenarios Codex must explicitly test

- A signed-in Player manually enters Captain and Studio URLs.
- A Player changes playthrough, Tale, chapter, player, invitation, or asset IDs in requests.
- A Player inspects page source, hydration state, network responses, prefetches, and browser caches for spoilers.
- A revoked invitation is redeemed from a previously opened tab.
- A removed Player continues using an old session or realtime connection.
- A Captain without Creator permission calls Studio APIs directly.
- A Creator without Captain assignment attempts to operate a live voyage.
- A user replays a captured mutation request after the resource state has changed.

07 PLAYER TALL TALE LIBRARY

The Player Library is a first-class destination and lasting personal archive, not a thin list of launch links.

Library content model

Every Player must be able to see all Tall Tales to which they have been invited, added, or granted access. The library must group items by playthrough state while still allowing multiple invitations or playthroughs for the same Tall Tale. A Tall Tale is the authored work; a playthrough is one historical voyage through a specific version.

Library group	Included records	Primary action
Invitations	Pending invitations not yet accepted or declined	View Invitation / Accept / Decline
Awaiting Captain	Accepted or assigned playthroughs not yet launched	Open Waiting Room
In Progress	Launched, paused, or actively playable voyages	Continue the Tale
Completed	Completed playthroughs grouped by Tall Tale and run	Open Voyage Archive
Replay / New Edition	Only replay or newer-version opportunities explicitly offered to this Player	Accept Replay / Begin New Voyage
Expired or Revoked	Optional history view when useful and safe	View status; no access to protected Tale content

Tall Tale card requirements

- **Artwork:** use the approved Player-safe cover or invitation artwork. Never use unrevealed chapter art as a card background.
- **Identity:** Tall Tale title, spoiler-safe subtitle, Captain display name, and voyage/playthrough label.
- **State:** invited, awaiting Captain, in progress, paused, completed, expired, or revoked. Status must come from backend state.
- **Progress:** show only player-safe progress such as chapter count revealed, current revealed chapter, or percentage derived from allowed checkpoints. Never reveal future chapter names.
- **History:** for completed runs, show completion date, exact version label, ending/route only when revealed, and number of memories collected.
- **Action:** one clear primary action plus a restrained overflow menu for safe secondary actions.

Completed Tall Tales and historical versions

Completed Tall Tales must remain in the Player Library permanently unless the Player chooses to hide them from their default shelf. Do not overwrite the completed record when the Creator publishes a later version. Each completed playthrough must open a Voyage Archive bound to the exact version snapshot that was played.

- Group multiple completed playthroughs under the same Tall Tale while preserving each run separately.
- Display the published version label and publication date used for that run.
- Allow the Player to review only the historical content actually revealed during that playthrough, plus post-completion content intentionally released by the Captain or Creator.
- Preserve collected photos, artifacts, journal pages, clues, decisions, side quests, hint history when player-visible, ending, completion date, and optional Captain epilogue.
- If a later edition exists, do not silently replace the archive. Show a New Edition Available state only when the Player has been invited, assigned, or explicitly granted replay access.
- If the same Tall Tale is replayed, create another playthrough record rather than resetting or mutating the completed one.

HISTORICAL TRUTH

A Player's completed archive is evidence of what they actually experienced. Treat it as an immutable historical record, not a dynamic view of whatever the Creator happens to be editing today.

08 PLAYER LIBRARY EXPERIENCE DETAILS

The Player Library must remain beautiful, clear, spoiler-safe, and practical on desktop and phone.

Default presentation

Use a rich gallery view by default, with an optional compact list view when the library grows. Cards should feel like journals, sealed letters, map cases, or archive volumes, but the visual metaphor must never reduce readability or make status difficult to identify.

Visual states

State	Visual treatment	Required truth signal
Invited	Sealed letter or closed journal with sender seal	Invitation expiration and acceptance state
Awaiting Captain	Closed journal with subtle living ambience	Accepted and ready; Captain has not launched
In Progress	Open journal, bookmark, active compass, subtle reflected light	Current player-safe checkpoint and last synchronized time
Paused	Bookmark or tied ribbon, reduced motion	Who paused it and whether the Player may continue
Completed	Bound archive volume or keepsake box	Completion date and exact Tale version
Revoked/Expired	Dimmed but legible, no false danger styling	Access is no longer valid and why, when safe to disclose

Player filters and search

- Filter by Invited, Awaiting Captain, In Progress, Completed, and Hidden/Archived.
- Search by Tall Tale title, Captain display name, voyage label, completion year, or player-safe tag.
- Sort by recently active, recently invited, completion date, title, or manually pinned favorites.
- Permit hiding a completed Tale from the main shelf without deleting its historical record.
- Do not show Creator-only tags, internal status names, draft identifiers, or unrevealed location metadata in search indexes.

Empty states

- **No Tall Tales yet:** explain how invitation links and codes work, with a single Enter Invitation Code action.
- **No active voyages:** show completed memories or pending invitations before suggesting that there is nothing to do.
- **No completed voyages:** use encouraging world-consistent copy without inventing progress.
- **Offline or stale data:** show last successful synchronization time and make it explicit that status may be outdated.

Player privacy

A Player should see their own Tall Tales and crews according to membership. Do not expose other players' private profiles, email addresses, invitation tokens, device information, or Captain operational notes. For crew Tales, show only the crew identity fields explicitly approved for Player display.

09 INVITATION ACCEPTANCE AND PLAYER WAITING ROOM

The invitation journey should feel ceremonial while remaining fast, secure, and recoverable.

Invitation landing page

Opening a valid invitation link should focus the gateway on the Player path. Reveal a themed invitation page with Tall Tale artwork, spoiler-safe description, Captain identity, scheduled start information, and a clear acceptance action. Do not expose the raw token in rendered copy, analytics, or support screenshots.

1. Validate the invitation token server-side before returning Tale metadata.
2. Display the invitation recipient name when known, with a safe correction path if the Captain misspelled it.
3. Request the configured PIN/password only after the invitation identity is established.
4. Allow the Player to sign in to an existing account or continue as a guest profile if the invitation permits it.
5. Record acceptance atomically and prevent duplicate membership creation on refresh or double-click.
6. Add the playthrough to the Player Library immediately.
7. Enter the waiting room when the voyage is not yet launched.

Waiting room

- Show a closed journal or sealed voyage object with ambient motion.
- Display truthful readiness state: accepted, awaiting other players, awaiting Captain, scheduled for a specific time, or paused before launch.
- Offer audio test, accessibility preferences, device readiness, and a reconnect action.
- Do not expose chapter count, future locations, internal readiness issues, or Captain controls.
- When the Captain launches the voyage, transition into the journal opening sequence through realtime update or a short polling fallback.
- If realtime connection fails, show a visible reconnect state and allow manual refresh without losing acceptance.

Decline, expiration, and revocation

Players must be able to decline an invitation without deleting their account. Expired or revoked invitations must never be redeemable, but the Player may see a safe explanatory state and request a replacement through an approved channel. The system must distinguish expired, revoked, already used, invalid, and unauthorized without leaking sensitive Tale existence to arbitrary callers.

10 CAPTAIN LIBRARY AND COMMAND WORKSPACE

The Captain Library is the operational center for running voyages, not the authoring interface.

Captain Library groups

Group	Purpose	Important signals
Needs Attention	Voyages requiring a Captain action	Player waiting, clue request, checkpoint reached, connection issue
Active Voyages	Currently launched playthroughs	Current checkpoint, last player activity, pending reveal, pause state
Ready to Launch	Accepted and configured playthroughs	Player readiness, version, schedule, validation state
Invitations	Pending, viewed, accepted, expired, revoked, replaced	Recipient, delivery status, expiration, last activity
Published Tales	Available Tale versions for new voyages	Latest published version, active run count, readiness
Completed Playthroughs	Historical runs	Player/crew, version, completion date, ending, Captain interventions
Drafts	Optional shortcut into Studio	Draft health, unpublished changes, validation problems

Active voyage card

- Tall Tale and voyage name.
- Player or crew identity.
- Exact published version.
- Current player-safe checkpoint and current Captain-facing operational checkpoint.
- Last activity and synchronization health.
- Pending Player request or Captain action.
- Primary Open Captain Console action.
- Safe Preview as Player action that cannot mutate the real Player session.
- Pause, resume, send hint, reveal, and advance actions only when valid for the current state.

Captain details page

The Captain-facing Tall Tale details page should summarize published versions, active voyages, invited players, completed runs, operational readiness, and shortcuts into Studio. It must not mix draft editing controls directly into the live Captain console.

SEPARATION OF CONCERNS

Captain runs the performance. Creator builds the stage. The same human may hold both roles, but the interfaces must remain distinct so live operation is not surrounded by dangerous authoring controls.

11 CLEAN CAPTAIN INVITATION WORKFLOW

Inviting a Player must be one of the most polished flows in the platform.

Entry points

- Invite Players from a published Tall Tale card.
- Start New Voyage from a Tall Tale details page.
- Invite from the Captain Library global action.
- Duplicate a completed playthrough configuration into a new invitation without copying historical progress.

Invitation wizard

Step	Required fields and decisions
1. Select Tale	Tall Tale, published version, starting point, solo/crew, normal/test voyage.
2. Configure Voyage	Voyage name, Captain, progression mode, hints, side quests, schedule, accessibility defaults.
3. Add Players	Existing Player, new guest profile, email when supported, display name, crew role, optional per-player PIN.
4. Security	Expiration, redemption limit, account requirement, password/PIN, device/session policy.
5. Delivery	Secure link, QR code, short code, copyable message, email when configured.
6. Review	Player-safe preview, version lock, schedule, invitation summary, validation warnings.
7. Create	Atomic playthrough and invitation creation; then show management screen.

Voyage configuration

- **Voyage name:** human-readable and unique enough for Captain and Player libraries. Default from Player name plus Tale title but allow editing.
- **Published version:** required for real voyages. Once launched, the playthrough remains bound to that version.
- **Starting point:** normally the beginning; permit later chapter starts only when the published Tale and Captain policy allow it.
- **Progression mode:** Captain-controlled, rule-controlled, or hybrid. Existing manual behavior must remain supported.
- **Hints:** disabled, available on request, Captain-pushed, timed, or Tale-defined.
- **Side quests:** enabled according to published version and voyage configuration.
- **Schedule:** optional planned start time with explicit timezone and no false guarantee of Captain presence.
- **Replay mode:** creates a new playthrough while optionally carrying Player-safe preferences, never prior progress.

Review screen

Before creation, show exactly what the Player will see and flag any invalid or dangerous configuration. Prevent launch when the selected version is unpublished, missing required Player content, references inaccessible media, or contains unresolved story-graph errors.

12 INVITATION DELIVERY, LIFECYCLE, AND MANAGEMENT

Every invitation is a tracked security object, not merely a URL copied from the browser.

Delivery methods

Method	Phase-one requirement	Notes
Secure link	Required	Opaque token; copy button; themed invitation landing page.
QR code	Required	Encodes the secure link; themed printable/display card; fallback short code.
Short code	Required	Human-friendly code entered from Player gateway; rate-limited lookup.
Copyable message	Required	Captain can copy a polished invitation message containing the link and safe details.
Email	Optional initially	Add when outbound email infrastructure exists; do not block core invitation flow.

Invitation states

```

DRAFT
-> CREATED
-> SENT or COPIED
-> VIEWED
-> ACCEPTED
-> JOINED / READY
-> CONSUMED (when single-use policy applies)

Any non-final valid state may become:
-> EXPIRED
-> REVOKED
-> REPLACED

DECLINED is a player decision and must remain distinct from REVOKED.
```

Captain invitation dashboard

- Show recipient, Tale, voyage, published version, status, expiration, delivery methods, viewed time, accepted time, and replacement lineage.
- Provide Copy Link, Show QR, Copy Message, Extend Expiration, Revoke, Replace, and Open Player actions as permitted.
- Do not display raw tokens after creation. Regenerate/replace instead of revealing stored secrets.
- A replacement invitation must invalidate the original according to policy and preserve the audit relationship between them.
- Bulk invitations for crews must produce individual invitation identities unless the Captain deliberately chooses a shared crew link with clearly weaker accountability.

Token requirements

- Generate cryptographically secure random tokens with enough entropy for public links.
- Store only a one-way hash of the token, plus a non-secret lookup prefix if needed.
- Support expiration, revocation time, accepted time, redemption count, maximum redemptions, and last validation time.
- Bind the invitation to Tall Tale, published version, playthrough, intended player/crew, and allowed scope.
- Do not allow changing the bound version after acceptance without creating a new invitation or explicit safe migration flow.

13 CREATOR WORKSPACE AND STUDIO INTEGRATION

Creator is a distinct workspace even when the same account is also the Captain.

Creator entry

Selecting Creator on the gateway must authenticate the account and open the Studio Library when the account has Creator permission. A Captain/Creator account may switch between Captain's Command and Tall Tale Studio through an authenticated workspace switcher. The Player interface must not expose that switcher.

Studio Library

Group	Contents	Primary action
Drafts	Unpublished working copies and incomplete Tales	Continue Editing
Ready to Publish	Validated drafts meeting publication rules	Review and Publish
Published	Tall Tales with one or more immutable versions	Open Details / Create New Draft
Needs Attention	Broken references, missing media, unsafe Player exposure, migration conflicts	Resolve Issues
Archived	Retired Tales retained for history	Restore or View History

Publishing rules

- Publishing creates an immutable Tall Tale version snapshot.
- Editing a published Tale creates or updates a working draft; it never mutates the published snapshot.
- A draft may be based on a prior version and must record that ancestry.
- Publication validation must check story graph integrity, required Player content, asset availability, access scopes, Captain controls, recovery paths, and Player-safe projections.
- Publishing must produce release notes or a structured diff summary.
- Active playthroughs remain on their bound version unless an explicit, audited migration is supported and safe.

Preview modes

- **Player preview:** uses a temporary preview principal and synthetic reveal state. It must be visually marked and must not mutate live progress.
- **Captain preview:** shows operational controls for test data only or a non-mutating simulation.
- **Version preview:** renders the exact immutable snapshot, not the current draft.

14 TALL TALE AND PLAYTHROUGH VERSIONING

Authored history and played history are separate and both must be preserved.

Core definitions

Entity	Meaning	Mutability
TallTale	Long-lived identity, ownership, public metadata, lifecycle	Mutable metadata under policy
TallTaleDraft	Current editable working copy	Mutable with revision tracking
TallTaleVersion	Published complete snapshot	Immutable
Playthrough	One voyage by one Player or crew	State progresses; identity persists
PlaythroughMembership	Player/Captain relationship to a playthrough	Mutable under authorization with history
PlaythroughEvent	Append-only state/history event	Immutable or correction-by-compensating-event
MemoryArchive	Player-facing completed historical projection	Immutable source data; presentation may evolve

Version lock behavior

1. A Captain selects an immutable TallTaleVersion while configuring the voyage.
2. The Playthrough stores that version ID before invitations are issued.
3. Invitation metadata and Player-safe previews are generated from that version.
4. Launching the voyage permanently locks the binding for normal operation.
5. Publishing a later version does not affect the active or completed playthrough.
6. A replay creates a new Playthrough bound to the selected version.
7. An exceptional migration must be explicit, audited, compatibility-checked, reversible where practical, and never silently rewrite completed history.

Version comparison

- Compare metadata, chapters, story blocks, branching conditions, media, artifacts, Captain instructions, Player content, recovery logic, and configuration defaults.
- Classify changes as added, removed, modified, moved, renamed, or access-scope changed.
- Flag changes that would break active playthrough compatibility.
- Allow restore by creating a new draft based on the selected historical version. Never reopen the immutable version for editing.
- Allow fork into a new Tall Tale identity while preserving provenance.

Player-visible historical version

The Player may see the version label and a completed archive reconstructed from the exact snapshot and reveal history. The Player does not automatically receive the Creator's full authoring diff, hidden content, internal version notes, or versions they never played or were never offered.

15 DOMAIN DATA MODEL

Adapt names to existing conventions, but preserve the relationships and invariants described here.

Primary entities

Entity	Key fields
Account	id, username/email, password_hash, status, created_at, last_login_at
AuthenticationMethod	id, account_id, type, provider_identifier, credential metadata
RoleAssignment	account_id, role, scope_type, scope_id, granted_by, granted_at, revoked_at
PlayerProfile	id, account_id nullable, display_name, status, preferences, claimed_at
TallTale	id, slug, title, owner, status, cover_asset_id, created_at, archived_at
TallTaleDraft	id, tall_tale_id, based_on_version_id, revision, content root, updated_by, updated_at
TallTaleVersion	id, tall_tale_id, version_label, snapshot, checksum, published_by, published_at
Playthrough	id, tall_tale_version_id, voyage_name, status, captain_mode, configuration, launched_at, completed_at
PlaythroughMembership	playthrough_id, player_profile_id/account_id, role, status, joined_at, removed_at
Invitation	id, playthrough_id, intended_player_id nullable, token_hash, short_code_hash, status, expires_at, max_redemptions
InvitationEvent	invitation_id, event_type, actor, timestamp, safe metadata
PlaythroughEvent	playthrough_id, sequence, event_type, actor, versioned payload, timestamp
RevealState	playthrough_id, content node/asset ID, reveal status, revealed_by, revealed_at
Asset	id, owner/scope, storage key, content type, checksum, visibility classification
AuditEvent	actor, action, resource, outcome, timestamp, correlation_id, safe metadata

Critical invariants

- [] Every real playthrough references exactly one immutable published TallTaleVersion.
- [] A completed playthrough cannot be reset in place; replay creates a new playthrough.
- [] Invitation acceptance cannot create duplicate active membership through retries.
- [] A Player Library query returns only records where the Player has a valid invitation, assignment, membership, or historical completed membership.
- [] A revoked Player cannot continue fetching protected playthrough data after session invalidation policy applies.
- [] A Player-safe response cannot include unrevealed content, even if the Player can infer or guess an internal identifier.
- [] An asset may not be fetched solely because its storage URL is known.
- [] Published version snapshot and checksum cannot change after publication.
- [] Audit and playthrough events remain ordered and attributable.

Snapshot strategy

Choose a snapshot representation suited to the current stack: normalized immutable version tables, a version-root plus immutable child records, or a validated serialized snapshot with indexes for queryable fields. The design must support complete historical rendering, diffing, integrity checks, and Player-safe projection without depending on the mutable draft tables.

16 STATE MACHINES

Do not allow arbitrary status strings. Encode legal transitions and enforce them in domain services and persistence constraints where practical.

Playthrough states

```
DRAFT_SETUP
-> INVITING
-> READY
-> SCHEDULED (optional)
-> ACTIVE
-> PAUSED
-> ACTIVE
-> COMPLETED
```

DRAFT_SETUP / INVITING / READY / SCHEDULED may become CANCELLED.
ACTIVE / PAUSED may become ABANDONED only through an explicit Captain action.
COMPLETED is terminal for progress mutation.

Membership states

```
INVITED -> ACCEPTED -> READY -> ACTIVE_MEMBER -> COMPLETED_MEMBER
INVITED -> DECLINED
INVITED / ACCEPTED / READY -> REMOVED
Any non-terminal membership may become SUSPENDED under policy.
```

Tall Tale lifecycle

```
DRAFT -> VALIDATING -> READY_TO_PUBLISH -> PUBLISHED
PUBLISHED -> NEW_DRAFT (based on published version)
PUBLISHED -> ARCHIVED
ARCHIVED -> RESTORED (creates active metadata state; versions remain immutable)
```

Transition enforcement

- Validate actor permission and current state in one transaction.
- Use optimistic concurrency or row/version locking to prevent double launch, double acceptance, and conflicting Captain actions.
- Record a playthrough or audit event for meaningful transitions.
- Return structured conflict responses when state changed since the client loaded.
- Make retry-safe mutations idempotent where network retries are expected.

17 BACKEND SERVICES AND API CONTRACTS

Use domain-focused services rather than scattering permission and state-transition logic through route handlers.

Recommended service boundaries

Service	Responsibilities
AuthenticationService	Sign-in, session rotation, logout, credential verification, trusted-session policy.
AuthorizationService	Role/scoped permission checks, Player-safe policy decisions, asset authorization.
TallTaleService	Metadata, details projections, lifecycle, archive/restore.
DraftService	Editable draft graph, autosave, validation, revision conflicts.
PublishingService	Validation, immutable snapshot, checksum, release notes, version creation.
PlaythroughService	Setup, launch, pause, resume, advance, complete, replay, historical archive.
InvitationService	Create, deliver metadata, validate, accept, decline, revoke, replace, expire.
PlayerLibraryService	Authorized grouped library projection and search.
CaptainLibraryService	Needs-attention and operational projections.
AssetService	Upload metadata, scoped access, signed delivery, historical version binding.
RealtimeService	Authorized playthrough channels, reconnect, event sequence recovery.
AuditService	Security and state-change event capture.

Representative API capabilities

```

GET    /api/player/library
GET    /api/player/tales/:taleId
GET    /api/player/playthroughs/:playthroughId
GET    /api/player/playthroughs/:playthroughId/archive
POST   /api/invitations/resolve
POST   /api/invitations/:token/accept
POST   /api/invitations/:token/decline

GET    /api/captain/library
POST   /api/captain/playthroughs
POST   /api/captain/playthroughs/:id/invitations
POST   /api/captain/playthroughs/:id/launch
POST   /api/captain/playthroughs/:id/pause
POST   /api/captain/playthroughs/:id/resume
POST   /api/captain/playthroughs/:id/actions/:action

GET    /api/studio/library
POST   /api/studio/tales/:id/drafts
POST   /api/studio/tales/:id/validate
POST   /api/studio/tales/:id/publish
GET    /api/studio/tales/:id/versions
GET    /api/studio/tales/:id/versions/compare

```

Response design

- Return explicit view models per workspace instead of raw ORM entities.
- Include stable status codes and machine-readable state, plus user-facing copy keys or safe messages.
- Include server timestamps and event sequence/version numbers for synchronization truth.

- Do not return data merely because the frontend currently ignores it.
- Use pagination for large libraries, invitation history, audit history, and completed playthroughs.

18 REALTIME SYNCHRONIZATION AND INTERFACE TRUTH

Captain and Player screens must agree about what has actually happened.

Realtime events

- Invitation viewed, accepted, declined, expired, revoked, and replaced.
- Player joined waiting room, disconnected, reconnected, or became ready.
- Captain launched, paused, resumed, advanced, revealed, sent hint, or completed the voyage.
- Player reached a checkpoint, requested a hint, acknowledged a reveal, or completed a required action.
- Version or configuration conflict requiring Captain attention.

Transport behavior

Use the existing realtime technology when present. Otherwise use WebSocket or server-sent events with authenticated channel subscription, plus a conservative polling fallback. Realtime connection membership must be authorized against the playthrough on every connection and reauthorization boundary.

Truthful status requirements

- Never display Updated just now continuously unless a real update occurred.
- Show last server-confirmed synchronization time, connection state, and pending local action when relevant.
- Use event sequence numbers or version tokens to detect missed updates and refetch authoritative state.
- Do not optimistically claim a Captain reveal or Player acceptance succeeded until the server confirms it.
- If a mutation is pending, say so explicitly. If it failed, restore the prior state and explain the failure.

INTERFACE TRUTH RULE

Animations may anticipate a confirmed state only after the backend commits it. The platform must never use visual confidence to disguise uncertain or stale data.

19 MOTION, SOUND, AND CINEMATIC TRANSITIONS

Animations are part of the product, but they must be state-driven, interruptible, and accessible.

Motion architecture

- Create named transition sequences for gateway arrival, role selection, sign-in transition, library arrival, invitation opening, waiting-room launch, journal opening, workspace switching, and voyage completion.
- Drive sequences from application state rather than arbitrary timeouts.
- Use a transition controller capable of cancellation, skip, reverse/return behavior, reduced-motion substitution, and cleanup on unmount/navigation.
- Preload only assets safe for the current user and current reveal state.
- Avoid long empty waits. Longer sequences must contain meaningful staged motion and allow a skip or fast-return path.

Audio behavior

- Respect browser autoplay policies and existing user audio preference.
- Provide visible mute/volume controls and remember preference appropriately.
- Use ambience and object sounds sparingly; do not make every hover clang like a hardware store collapsing.
- Stop or reduce ambience when the page is hidden or the user enters a content-heavy accessibility mode.

Reduced-motion equivalents

When reduced motion is enabled, replace large camera travel, parallax, page flips, and continuous drift with short crossfades, direct state changes, and restrained emphasis. Preserve hierarchy, confirmation, and story meaning without forcing motion.

20 RESPONSIVE DESIGN AND DEVICE BEHAVIOR

The platform must support desktop, laptop, tablet, and phone without treating mobile as a shrunk desktop.

Gateway responsiveness

- Desktop: role objects may share one cinematic scene and respond to pointer movement.
- Tablet: retain depth but reduce simultaneous effects and ensure touch targets are generous.
- Phone portrait: present role objects as a vertical cinematic sequence or carousel with clear labels; do not cram three tiny objects across the screen.
- Landscape phone: preserve safe-area insets and avoid role controls behind browser UI.

Library responsiveness

- Desktop: gallery plus optional list view, persistent filters where space allows.
- Phone: single-column cards, sticky state-aware primary action, bottom-sheet filters, and concise metadata.
- Keep completed historical archives readable on phone, including photos, journal pages, and timeline events.
- Do not hide critical state or authorization failures inside hover-only interactions.

Captain operating mode

Captain controls must be usable from a second monitor, laptop, or phone during gameplay. Prioritize clear current state, large safe action targets, confirmation for irreversible actions, and minimal navigation depth. Do not require the Captain to enter Studio to operate a live voyage.

21 ACCESSIBILITY

Cinematic presentation must not exclude keyboard, screen-reader, low-vision, motion-sensitive, or touch-only users.

- [] Every role choice is a semantic button or link with an accessible name and description.
- [] Keyboard focus order follows the visual order and remains visible against animated backgrounds.
- [] All dialogs trap focus correctly and return focus to the triggering control.
- [] Status is not communicated by color or animation alone.
- [] Invitation QR codes always have a human-readable code and link alternative.
- [] Forms have persistent labels, helpful errors, and no placeholder-only identification.
- [] Screen readers receive meaningful live-region updates for invitation acceptance, Captain launch, and synchronization state without excessive announcements.
- [] Reduced motion is honored automatically and can also be selected in application preferences.
- [] Text remains readable at 200% zoom and with browser font scaling.
- [] Color contrast meets WCAG AA for essential text and controls.
- [] Touch targets are at least the platform-appropriate accessible minimum.
- [] Historical photos and story images support authored alt text and decorative classification.

22 LOADING, ERROR, CONFLICT, AND RECOVERY STATES

The platform must behave gracefully when networks, sessions, humans, and browsers perform their customary acts of sabotage.

Loading

- Use scene-consistent skeletons or staged reveals, but do not show fake progress values.
- Load Player-safe library metadata before optional decorative assets.
- Keep sign-in and invitation redemption controls responsive while nonessential ambience loads.

Errors

Condition	Required behavior
Invalid/expired invitation	Safe explanation, no Tale leak, return to Player gateway, optional request-replacement guidance.
Unauthorized route	Generic access-denied or safe 404; no protected data in page source.
Session expired	Preserve safe intended destination and return to matching sign-in.
Version conflict	Explain that state changed; refetch; do not overwrite newer state.
Realtime disconnected	Show disconnected/stale indicator, retry, and authoritative refresh.
Asset unavailable	Use a designed fallback and log actionable diagnostics; do not expose storage internals.
Captain action failed	Do not animate success; restore controls; show retryable error with correlation ID when appropriate.

Recovery

- Invitation acceptance must be idempotent across refresh and repeated submission.
- Incomplete voyage setup must save as a resumable draft rather than creating orphaned records.
- If QR generation fails, the secure link and short code remain available.
- If Player launch synchronization is missed, the Player can refresh and enter the correct active state.
- If a Captain accidentally navigates away, the live voyage remains authoritative on the server and reopens to the current state.

23 AUDIT HISTORY AND OPERATIONAL DIAGNOSTICS

Meaningful access and story-control actions must be traceable without storing secrets or spoiling the Player experience.

Audit events

- Authentication success/failure, session creation, logout, role denial, and privilege change.
- Invitation created, copied/sent, viewed, accepted, declined, expired, revoked, replaced, and redeemed.
- Player added, removed, suspended, restored, or account-claimed.
- Playthrough created, configured, launched, paused, resumed, advanced, completed, abandoned, or replayed.
- Hint sent, reveal performed, checkpoint overridden, or progress corrected.
- Draft validated, version published, restored to draft, forked, archived, or unarchived.
- Protected asset access denial and suspicious code/token attempts when useful for security monitoring.

Audit design

- Record actor, action, resource, outcome, timestamp, correlation ID, and safe metadata.
- Never record raw passwords, PINs, invitation tokens, session cookies, or complete private story payloads.
- Provide Captain-visible operational history only for authorized Tales and voyages.
- Keep security diagnostics separate from Player-facing story history.
- Use retention appropriate to the project and document it.

24 MIGRATION FROM THE CURRENT APPLICATION

Preserve current Player, Captain, Studio, Tall Tale, and password behavior while moving to the unified model.

Repository discovery before migration

- [] Identify existing Player login/password records and how a Player is currently associated with a Tall Tale.
- [] Identify Captain authentication and whether Captain and Studio already share credentials.
- [] Map every current Tall Tale record, chapter, clue, media asset, progress field, and Captain control to the new domain model.
- [] Identify active sessions and current URLs that must redirect safely.
- [] Inventory existing animations and reusable visual assets.
- [] Identify whether current progress is mutable content state, event history, or both.

Backfill strategy

1. Create TallTale identities for all existing Tales.
2. Create an initial immutable published version from the current playable content of each Tale.
3. Create a working draft based on that version only when current unpublished edits exist.
4. Create PlayerProfile records for existing Player identities or password-bound Player entries.
5. Create Playthrough records for current active and completed Tall Tale progress.
6. Bind each migrated playthrough to the generated initial published version.
7. Create historical PlaythroughEvents or a clearly marked migration snapshot event sufficient to preserve current state.
8. Create RoleAssignments for existing Captain and Studio accounts.
9. Redirect old Player/Captain/Studio routes to the new route families while preserving intended destination.

Compatibility

- Existing Player passwords must continue to work during the transition unless intentionally reset through a visible migration flow.
- Existing active Tall Tales must resume at the same progress point.
- Existing Captain controls must not disappear before their replacements are functional.
- Do not expose migrated drafts or Captain notes to Players due to a broad serialization shortcut.
- Provide a rollback plan for database migration and a dry-run validation report.

25 IMPLEMENTATION PHASES

Build in coherent vertical slices so each phase produces a usable, testable platform increment.

Phase 1 - Domain foundation and access control

- [] Repository inventory and architecture decision record.
- [] Account, role assignment, Player profile, Tale, version, playthrough, membership, invitation, event, and asset-scope model changes.
- [] Server-side authorization service and protected route middleware.
- [] Initial Tall Tale version snapshot and migration/backfill tooling.
- [] Compatibility tests for current routes and passwords.

Phase 2 - Gateway and authentication

- [] Cinematic role gateway with Player, Captain, and Creator options.
- [] Player invitation/direct sign-in flow.
- [] Captain/Creator sign-in and authorized workspace selection.
- [] Session restore, logout, expiry, and safe redirects.
- [] Reduced-motion and responsive variants.

Phase 3 - Player Tall Tale Library

- [] Invited, awaiting Captain, active, paused, completed, and replay/new-edition groups.
- [] Card states, search, filters, list/gallery modes, empty states.
- [] Completed Voyage Archive bound to exact version.
- [] Multiple playthrough grouping and permanent historical access.
- [] Player-safe data and asset projections.

Phase 4 - Captain Library and invitations

- [] Captain Library groups and needs-attention projection.
- [] Playthrough setup and invitation wizard.
- [] Secure link, QR, short code, copyable message.
- [] Invitation dashboard, lifecycle actions, and audit history.
- [] Waiting room and Captain launch synchronization.

Phase 5 - Studio integration and publishing

- [] Creator gateway and Studio Library.
- [] Immutable publishing service, validation, release notes, version comparison.
- [] Preview modes and workspace switching.
- [] Restore-to-draft and fork operations.

Phase 6 - polish, diagnostics, and hardening

- [] Full cinematic transitions and sound behavior.
- [] Authorization penetration tests and asset-leak tests.
- [] Performance, offline/stale-state, reconnect, and concurrency testing.
- [] Accessibility audit and reduced-motion validation.
- [] Documentation, migration report, and final acceptance walkthrough.

26 TESTING STRATEGY

Tests must prove both beautiful journeys and ugly edge cases, because software mainly encounters the latter.

Unit tests

- Role and scoped authorization policies.
- Player-safe projection and field redaction.
- Legal/illegal state transitions.
- Invitation token hashing, expiration, revocation, replacement, and redemption limits.
- Version publication immutability and checksum validation.
- Library grouping and sorting.
- Replay creation without prior progress mutation.

Integration tests

- Database migrations and backfill against representative current data.
- Authentication plus route middleware.
- Invitation create-resolve-accept flow with retry/idempotency.
- Captain launch and Player realtime transition.
- Completed playthrough archive from immutable version and reveal history.
- Protected asset authorization with guessed IDs/URLs.
- Publishing while active playthroughs remain on prior version.

End-to-end journeys

1. New visitor selects Player, enters short code, accepts invitation, sees it in their library, waits, and enters when Captain launches.
2. Returning Player signs in, sees invitations, active voyage, completed historical runs, and opens the exact version archive.
3. Captain signs in, creates voyage, generates QR/link/code, sees invitation viewed and accepted, then launches.
4. Creator signs in, edits draft, validates, publishes a new version, and confirms existing playthrough remains unchanged.
5. Player manually enters a Captain URL and receives no protected data.
6. Player guesses future asset and chapter IDs and receives no content.
7. Two Captain devices attempt conflicting actions and receive correct concurrency handling.
8. Invitation is revoked while open and cannot be accepted afterward.

Visual and accessibility regression

- Desktop and phone screenshots for every major state.
- Keyboard-only gateway, login, library, invitation, and Captain flow.
- Screen-reader labels and live announcements.
- Reduced-motion snapshots and transition behavior.
- High zoom, long titles, missing art, many completed runs, and localization-length stress cases.

27 ACCEPTANCE CRITERIA

The work is complete only when the following observable outcomes are true.

Gateway

- [] The first page presents a polished game-like selection for Player, Captain, and Creator, or a documented two-role presentation with Creator accessible after Captain authentication.
- [] Selecting a role changes the sign-in path but never grants authorization.
- [] Returning sessions receive a truthful, fast continuation path.
- [] The gateway is responsive, keyboard accessible, and reduced-motion compatible.

Player Library

- [] A Player can see every pending invitation and every Tall Tale to which they were added.
- [] A Player can distinguish awaiting-Captain, in-progress, paused, completed, expired, and revoked states.
- [] A Player can reopen completed Tall Tales and view each separate completed playthrough.
- [] Each completed playthrough displays and renders from the exact published Tall Tale version used for that run.
- [] Publishing a new version does not change any existing active or completed Player archive.
- [] The Player receives no Captain, Creator, future chapter, hidden asset, or unrevealed story data.

Captain invitations

- [] A Captain can create a playthrough from a published Tall Tale and select the version.
- [] A Captain can invite an existing or new Player using a secure link, QR code, and short code.
- [] Invitation status accurately progresses through viewed, accepted, ready, expired, revoked, or replaced.
- [] A Captain can revoke or replace an invitation without exposing stored tokens.
- [] A Player accepting an invitation is added to their library exactly once, even after refresh or retry.
- [] The Player waiting room transitions to the Tale only after an authoritative Captain launch.

Creator and versions

- [] Creator access is separately authorized even when the account is also a Captain.
- [] Publishing creates an immutable version and does not mutate previous versions.
- [] Version comparison, restore-to-draft, and fork behavior preserve ancestry.
- [] Player preview and Captain preview cannot mutate live playthrough progress.

Security and quality

- [] Protected routes, APIs, assets, exports, and realtime channels are server-authorized.
- [] No raw password, PIN, invitation token, or session secret is logged or stored improperly.
- [] Interface timestamps and synchronization states reflect real backend events.
- [] Automated tests cover authorization denial, token lifecycle, version lock, and historical archive behavior.
- [] No placeholder controls, dead navigation, fake statuses, or unfinished invitation actions remain.

28 CODEX EXECUTION CHECKLIST

Use this as the final implementation sequence and completion report structure.

Before editing

- [] Read repository instructions and project documentation.
- [] Inspect current architecture, routes, auth, data model, tests, build commands, and visual system.
- [] Record current behavior and migration risks.
- [] Identify the smallest safe sequence of vertical slices.

During implementation

- [] Create migrations and rollback/dry-run validation.
- [] Implement domain services and authorization before wiring protected UI.
- [] Build Player-safe view models and asset delivery.
- [] Implement gateway, libraries, invitation flow, versioning, and realtime behavior.
- [] Preserve current working functionality through adapters and redirects.
- [] Add tests with each vertical slice rather than at the end.
- [] Keep design consistent with the existing Tall Tale visual language.

Before declaring completion

- [] Run formatters, linters, type checks, unit tests, integration tests, and end-to-end tests.
- [] Exercise all role journeys manually on desktop and phone-sized viewports.
- [] Verify Player cannot obtain Captain/Creator or spoiler data through URLs, APIs, assets, preload state, exports, or realtime channels.
- [] Verify current Tall Tales, Player passwords, Captain access, and progress survived migration.
- [] Verify completed playthroughs remain bound to historical versions after publishing a new version.
- [] Update documentation and provide a concise implementation report including files changed, migrations, tests, remaining risks, and screenshots where the project workflow supports them.

COMPLETION STANDARD

Do not report the feature as complete because the pages exist. It is complete when the entire invitation-to-library-to-launch-to-completion-to-historical-archive journey works, permissions hold under hostile navigation, and the interface remains beautiful without lying about state.

29 REFERENCE USER JOURNEYS

These journeys define the intended product behavior in plain sequence form.

Journey A - Invited Player

1. The Captain creates Sera's Forever Treasure voyage from published version 1.0.
2. The Captain generates a themed invitation link, QR card, and short code.
3. The Player opens the link and sees the Player-focused invitation gateway.
4. The Player confirms identity and enters the configured password or PIN.
5. The invitation is accepted exactly once and appears in the Player Library.
6. The Player enters an Awaiting Captain waiting room.
7. The Captain sees the Player ready and launches the voyage.
8. The Player receives the journal-opening transition and enters the active Tale.

Journey B - Returning Player with history

1. The Player signs in through the Player path.
2. The Player Library shows one pending invitation, one active Tale, and two completed playthroughs.
3. The Player continues the active Tale without seeing future chapter details.
4. The Player opens a completed Tall Tale and selects the July 2026 playthrough.
5. The Voyage Archive renders from version 1.0 and the exact reveal history of that run.
6. A newer version 1.2 is not shown unless the Player has been offered or invited to it.

Journey C - Captain creates and operates

1. The Captain selects Captain at the gateway and signs in.
2. Captain's Command shows a needs-attention card for a waiting Player.
3. The Captain opens another published Tale and starts a new voyage.
4. The invitation wizard validates version, Player, security, and delivery.
5. The Captain copies the message and displays the QR code.
6. Invitation status updates from viewed to accepted to ready.
7. The Captain launches and later advances the Tale through authoritative controls.

Journey D - Creator publishes without rewriting history

1. The Creator selects Creator and signs in with the same account used for Captain access.
2. The Studio opens the working draft based on version 1.0.
3. The Creator adds a side quest, validates the story graph, and publishes version 1.1.
4. Existing active and completed playthroughs remain on version 1.0.
5. New voyages may select version 1.1.
6. A Player sees a New Edition invitation only when the Captain creates one.

30 FINAL BUILD DIRECTIVE

This is the concise command that governs the entire implementation.

BUILD DIRECTIVE

Implement a unified Tall Tale platform in the existing repository with a cinematic Player/Captain/Creator entry gateway, secure role-aware authentication, a complete Player Tall Tale Library containing invitations, assigned Tales, active voyages, completed voyages, and exact historical versions, a Captain Command library with polished invitation creation and management, a separate Creator Studio workspace, immutable published versions, version-locked playthroughs, realtime launch/progress synchronization, spoiler-safe data projections, protected assets, responsive cinematic design, accessibility, migrations, tests, and documentation. Preserve all current working behavior and data. Do not substitute hidden buttons for authorization, dynamic current drafts for historical truth, or decorative status text for real backend state.

Definition of success

A Player should feel as though they entered a living collection of adventures. A Captain should feel as though they entered a clear command center. A Creator should feel as though they entered a serious story-building studio. The database and backend should quietly enforce identity, history, state, and secrecy underneath all three, because apparently even enchanted pirate journals require competent systems engineering.